Real Time Fraud Detection Model

Detailed Documentation of Model Building Process

Prepared by: Xueshan (Kevin) Peng

4/3/2026

Table of Contents

1. **Executive Summary**
2. **Description of the Data**
    - Data Overview
    - Summary Statistics
    - Key Insights
3. **Data Cleaning**
    - Data Exclusions
    - Outlier Treatment
    - Imputation Methods
4. **Data Partitioning**
    - Option 1: Random Split with K-Fold Cross-Validation
    - Option 2: Strict Time-Series Split
    - Option 3: Hybrid Historical K-Fold with Out-of-Time (OOT) Validation
    - Selected Approach
5. **Variable Creation**
    - Entity Construction
    - Target Encoded Features
    - Time-Based Aggregation Features
    - Velocity and Recency Ratio Features
    - Amount Variability Features
    - Cross-Entity Diversity Features
    - Business Logic Features
6. **Feature Selection**
    - Methodology

- Final Variable List

7. **Preliminary Model Exploration**
    - Model Descriptions
    - Comparative Performance Table
8. **Final Model Evaluation**
    - Final Model Architecture and Hyperparameters
    - Final Model Performance
    - Results: Training Set, Testing Set, Out-of-Time (OOT) Set
9. **Financial Curves and Recommended Cutoff**
    - Financial Impact Plots
    - Recommended Cutoff Strategy
10. **Summary**
11. **Appendix**
    - Full Data Quality Report (DQR)

Executive Summary

A predictive fraud model was developed to proactively identify and stop fraudulent card transactions. This involved analyzing historical transaction data to build behavioral and velocity-based features that can detect spending anomalies. The final model demonstrates strong, stable performance on future, out-of-time (OOT) data.

The model's business value is significant. It is highly efficient, achieving a **Fraud Detection Rate (FDR) of 62.64% while reviewing only the top 3%** of the highest-risk transactions. Based on the full financial analysis, the recommended implementation strategy, flagging the top 5% of transactions, is projected to generate **$54.2 million in net savings**. This approach optimally balances maximizing fraud capture with managing operational costs and ensuring a positive customer experience.

Description of the Data

Data Overview

The dataset is **Card Transactions Data**, which includes merchant identifiers, transaction types, and a binary fraud label. The data came from around 100,000 real U.S. applications **over 8 months**. There are **10 fields** and **98,393 records**.

Summary Statistics

Below is a summarization of all the variables in the original dataset.

Fig 1a. Numeric Fields Table

Field Name	Field Type	# Records Have Values	% Populated	# Zeros	Min	Max	Mean	Standard Deviation	Most Common
Amount	numeric	98393	100.00%	0	0.01	3102046	424.2909	9922.44	3.62

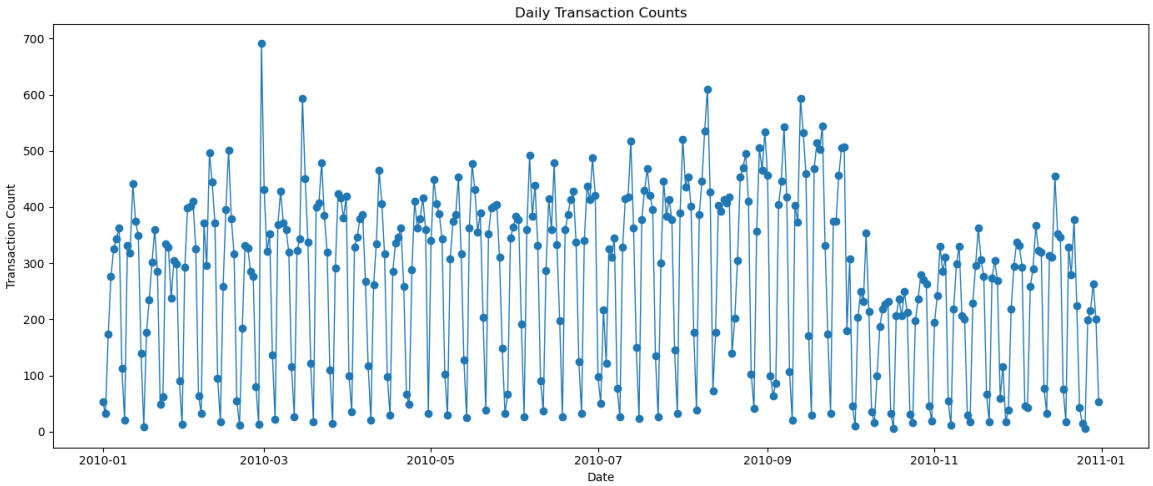
Fig 1b. Categorical Fields Table

Field Name	Field Type	# Records Have Values	% Populated	# Zeros	# Unique Values	Most Common
Date	Categorical	98393	100.00%	0	365	2/28/10
Merchnum	Categorical	94970	96.52%	0	13091	930090121224
Merch description	Categorical	98393	100.00%	0	13126	GSA-FSS-ADV
Merch state	Categorical	97181	98.77%	0	227	TN
Transtype	Categorical	98393	100.00%	0	4	P
Recnum	Categorical	98393	100.00%	0	98393	1
Fraud	Categorical	98393	100.00%	95901	2	0
Merch zip	Categorical	93664	95.19%	0	4567	38118
Cardnum	Categorical	98393	100.00%	0	1645	5142148452

Key Insights

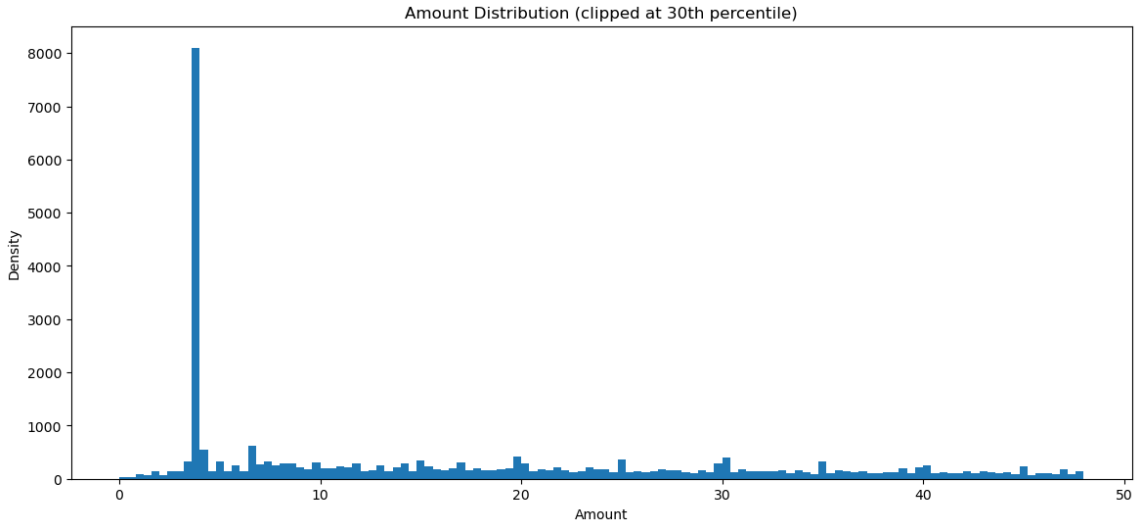
Transaction volume exhibits a strong weekly pattern. A significant decline in transactions occurs around October, which aligns with the renewal of the fiscal year. This suggests that business spending increases before the renewal and becomes more conservative immediately after.

Fig 2a. Daily Transaction Counts



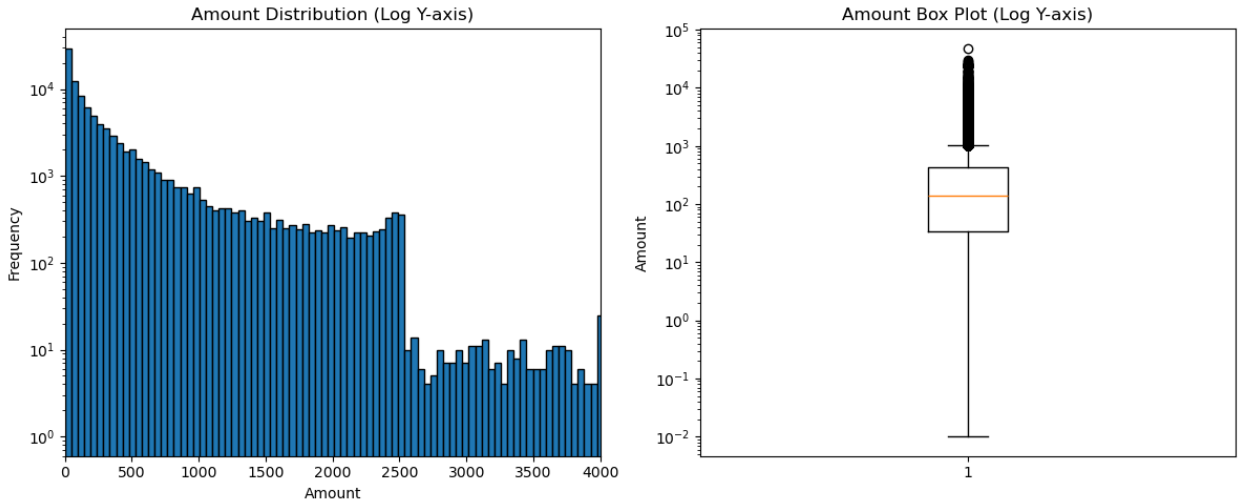
A notable anomaly was detected in the transaction data: a high-frequency spike at the **$3.62** price point. Investigation confirms these transactions primarily consist of uniform charges from **FedEx**.

Fig 2b. Transaction Amount Distribution (Clipped at 30th Percentile)



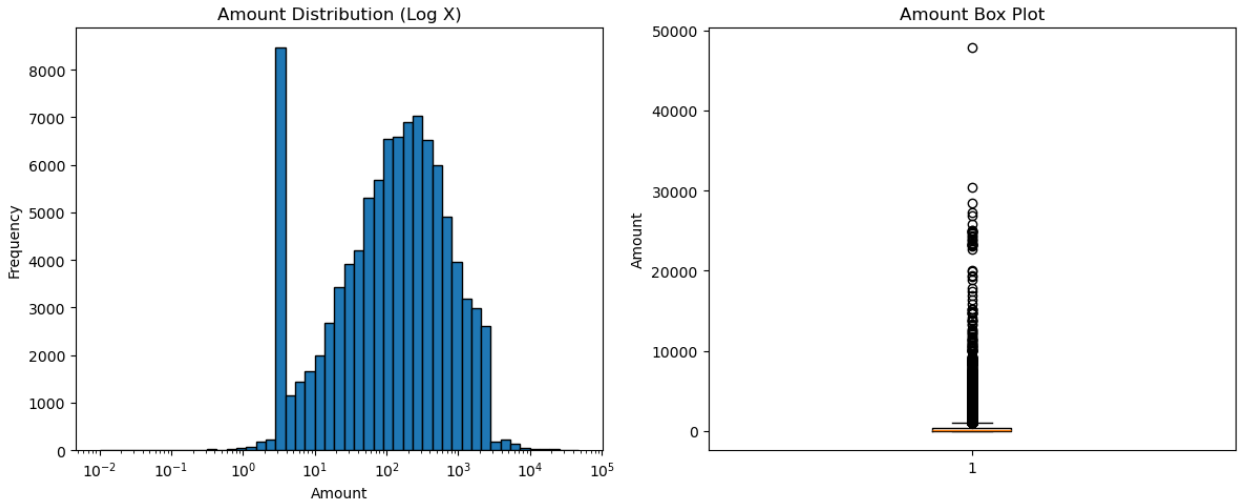
Transaction amounts show a sharp decline at the $2,500 mark. This pattern is consistent with most companies having an internal policy requiring manager approval for expenditures above this threshold.

Fig 2c. Transaction Amount Distribution and Amount Box Plot (Log Y-axis)



Transaction amounts also contain an appreciable number of outliers, as shown in *Fig 2d. Transaction Amount Distribution and Amount Box Plot*.

Fig 2d. Transaction Amount Distribution and Amount Box Plot



Data Cleaning

Generally, the data cleaning methodology consists of three components:

1. **Exclusion**: Excluding records that do not align with the target population to ensure the model remains relevant.
2. **Outlier**: Limiting, rather than removing, the influence of outliers. Because these data points represent valid members of the population, they should be retained but managed, via flagging or winsorization (capping values at n standard deviations), to prevent them from skewing the model.
3. **Imputation**: Estimating missing values based on available fields. While some models handle missingness automatically, manual imputation allows the developer to apply domain expertise, ensuring the model receives the clearest possible signals for learning.

Data Exclusions

There are also four different transaction types:

- Purchases (98038 records)
- A (181 records)
- Declined (173 records)
- Y (1 record)

For our purposes, we are only interested in purchases. All other transactions are excluded.

Transactions with value (`Amount`) over $3 Million dollars will also be excluded.

Outlier Treatment

A rule-based outlier flag, `amount_okay`, was created to identify and isolate a known set of anomalous transactions. Transactions were flagged as anomalous (False) if their `Merch description` began with "FEDEX" and the transaction `Amount` was between $3.62 and $3.80, inclusive. All other transactions were flagged as True (okay). Please refer to *Fig 2b. Transaction Amount Distribution (Clipped at 30th Percentile)* for visualization of the distribution of `Amount`.

Imputation Methods

Missing values were systematically addressed across three key merchant-related fields: Merchant Number (`Merchnum`), Merchant State (`Merch state`), and Merchant Zip Code (`Merch zip`). A cascading imputation strategy was employed for each variable, leveraging known relationships among existing fields to maximize the accuracy and completeness of imputed values. Remaining records that could not be resolved through any available mapping were assigned a placeholder value of "unknown" to preserve data integrity and prevent information loss through row deletion.

Merchant Number

Merchant Number values recorded as "0" were first recoded to null to ensure consistency in missing value representation. Imputation then proceeded in three stages:

1. **Description-Based Lookup:** A mapping was constructed from existing records where both the Merchant Description and Merchant Number were non-null. For each unique merchant description, the first observed Merchant Number was retained. Missing Merchant Number values were then populated by matching on the merchant description.
2. **Adjustment Transaction Classification:** Transactions identified as "Retail Credit Adjustment" or "Retail Debit Adjustment" are non-merchant transactions by nature and were assigned a Merchant Number value of "unknown."
3. **Synthetic Identifier Assignment:** For records where the Merchant Number remained null after the preceding steps, a set of synthetic identifiers was generated. Sequential numeric values, beginning from one increment above the maximum existing Merchant Number, were assigned to each unique remaining merchant description to ensure referential uniqueness.

Merchant State

Missing Merchant State values were imputed through a multi-layered approach:

1. **Zip Code-to-State Mapping:** A lookup table was constructed from records containing both valid zip codes and state values. This mapping was supplemented with manually verified entries for specific zip codes—predominantly those associated with Puerto Rico (PR) and the U.S. Virgin Islands (VI)—that could not be resolved through the automated mapping alone.

2. **Merchant Number-to-State Mapping:** For records still lacking a state value, a secondary mapping was applied using the Merchant Number field, drawing from the first observed state associated with each merchant.
3. **Merchant Description-to-State Mapping:** A tertiary mapping was constructed from merchant descriptions to further resolve remaining null values.
4. **Adjustment Transaction Classification:** Adjustment transactions ("Retail Credit Adjustment" and "Retail Debit Adjustment") were assigned a state value of "unknown."
5. **Foreign Classification:** State values that did not correspond to any of the 50 U.S. states, the District of Columbia, Puerto Rico, or the U.S. Virgin Islands were reclassified as "foreign."
6. **Final Default Assignment:** Any remaining null values were assigned "unknown."

Merchant Zip Code

Missing Merchant Zip Code values were imputed using a similar cascading strategy:

1. **Merchant Number-to-Zip Mapping:** A mapping was constructed from records with valid Merchant Number and zip code pairs. Missing zip codes were populated based on the first observed zip code for each merchant.
2. **Merchant Description-to-Zip Mapping:** A secondary mapping using merchant descriptions was applied to resolve additional null values.
3. **Adjustment Transaction Classification:** Adjustment transactions were assigned a zip code value of "unknown."
4. **State-Based Fallback:** For records still lacking a zip code, the most populous zip code for the corresponding state was assigned as a representative proxy.
5. **Final Default Assignment:** Any remaining null values were assigned "unknown."

Following imputation, all zip codes were standardized to five-digit format through left-padding with leading zeros where applicable, ensuring consistent formatting across the dataset. Whitespace was also removed from all merchant description values to support downstream matching and feature engineering.

Data Partitioning

Before starting the discussion on feature construction, data partitioning is an important subject to address.

For the purposes of this analysis, the following terminology is adopted to describe the data partitions:

- **Training Set** — The subset of data on which the model is trained.
- **Testing Set** — The subset used for immediate model evaluation, informing iterative improvements to model architecture and hyperparameters.
- **Validation Set (Out-of-Time)** — A held-out subset intentionally reserved as a final, unbiased evaluator of the fully tuned model.

Three candidate partitioning strategies were considered:

Option 1: Random Split with K-Fold Cross-Validation

This approach treats all transactions as independent observations, disregarding temporal ordering. While it maximizes data utilization and yields stable variance estimates across folds, it introduces a significant risk of temporal leakage and entity-level overfitting. Specifically, the model may learn patterns from fraudulent actors whose transactions appear in both the training and testing folds, resulting in artificially inflated performance metrics that do not generalize to production conditions.

Option 2: Strict Time-Series Split

Under this strategy, the data is partitioned chronologically using a walk-forward validation scheme (e.g., training on Month 1 and testing on Month 2). This method faithfully simulates model degradation as fraud tactics evolve over time. However, early folds suffer from severely limited training data—a critical limitation in fraud detection where the positive class is inherently rare. Furthermore, by framing the problem as a forecasting exercise, this approach may unfairly penalize the model by restricting access to the full breadth of historical behavioral patterns that would be available during real-time inference.

Option 3: Hybrid Historical K-Fold with Out-of-Time (OOT) Validation

This approach reserves the most recent period of data as a temporally isolated validation set, while applying K-Fold cross-validation to the remaining historical data for training and testing. It combines the statistical robustness of K-Fold resampling with the temporal integrity of an out-of-time holdout for final model sign-off. The primary requirement is strict feature engineering discipline: all time-dependent aggregates (e.g., 30-day cumulative spend) must be computed using only data available prior to each transaction's timestamp to prevent information leakage.

Selected Approach

Option 3 was selected as the most appropriate strategy for this study. It provides sufficient training data for reliable model estimation while preserving a temporally clean validation set that reflects realistic deployment conditions. To uphold the integrity of this design, all engineered features were computed using a strictly backward-looking, expanding historical window, ensuring that no future information was incorporated into any transaction's feature representation. No data beyond the timestamp of the transaction would be available when the feature is calculated.

One notable limitation of this study is that the dataset spans only a single calendar year. Ideally, the training partition would encompass a full year of data to allow the model to learn seasonal patterns across all periods. However, reserving a temporally isolated out-of-time holdout necessarily reduces the training window, and this trade-off was accepted in order to maintain a data-leak-proof evaluation design. As a result, model performance on the out-of-time holdout is expected to be conservatively understated relative to what would be achieved in a production environment with a longer historical training period.

Variable Creation

To enhance the model's predictive power, the raw, transaction-level data was engineered into a comprehensive set of features. The objective was to transform raw transactional records into a rich set of behavioral and contextual signals that reflect both the historical patterns of individual entities and the characteristics of each transaction relative to those patterns. All features were constructed using a strictly backward-looking methodology, ensuring that only data available prior to each transaction's timestamp was incorporated. This discipline was maintained throughout to prevent any form of information leakage from future observations.

Entity Construction

To capture behavioral patterns across multiple levels of transaction context, a set of composite entity keys was derived by concatenating base fields. Two-way and three-way combinations were formed across merchant number, merchant description, zip code, state, and day of week. The final retained entity set comprised 26 distinct grouping dimensions used as the basis for all time-based aggregations.

Target Encoded Features

Three categorical variables — Merchant State, Merchant ZIP Code, and Day of Week — were converted to numeric representations using Bayesian smoothed target encoding. Standard library implementations were not employed, as they do not support a historically expanding evaluation window, which is essential for leakage-free encoding. A custom implementation was developed for this purpose.

For each category, a smoothed fraud rate estimate was computed as described in the following *Fig 3. Target Encoding Methodology and Definition*.

Fig 3. Target Encoding Methodology and Definition

We define a weighting factor as (w):

$$w = \frac{n}{n+k}$$

(Where n is the Total Count and k is the Smoothing Factor)

The smoothed estimate is:

$$\text{Smoothed Value} = w \cdot (\text{Category Mean}) + (1-w) \cdot (\text{Global Mean})$$

If we substitute the Category Mean as $\frac{\text{Fraud Count}|\text{Category}}{n}$ and simplify the algebra, we arrive at the following formula:

$$\frac{(\text{Fraud Count} \mid \text{Category}) + (\text{Global Mean} \times k)}{n+k}$$

Time-Based Aggregation Features

For each of the 26 entity dimensions, the following features were computed across rolling time windows of 0, 1, 3, 7, 14, 30, and 60 days prior to each transaction:

- **Days since last transaction** — The number of elapsed days since the entity was last observed. For entities with no prior transaction history, this was set to the number of days since January 1, 2010, serving as a proxy for a "new" entity.
- **Transaction frequency** — Count of prior transactions within each time window.
- **Amount statistics** — Mean, maximum, median, and cumulative total transaction amount within each window.
- **Amount ratio features** — The ratio of the current transaction amount to the historical mean, maximum, median, and cumulative total within each window. These capture whether the current transaction is anomalously large or small relative to past behavior.

Velocity and Recency Ratio Features

Velocity ratios are calculated to measure the intensity of recent transaction activity relative to an entity's long-term historical average. By comparing short-term windows (0–1 days) against extended periods (7, 14, 30, and 60 days), the system identifies sudden "bursts" of activity that often precede fraudulent events.

- **Transaction Count Velocity**: This ratio compares the frequency of transactions in the immediate term to the average frequency over a longer horizon. A significant spike in this ratio indicates a high-frequency usage pattern that deviates from the norm.
- **Amount Velocity**: This compares the total monetary value spent in the short term against the historical average. This is critical for detecting "spending sprees" where the volume of funds being moved increases rapidly.
- The **Velocity-to-Recency ratio (VDR)** adds a temporal decay component to the velocity metrics. It normalizes the current activity rate by the time elapsed since the last recorded transaction. A high VDR occurs when an account that has been inactive for a significant period (high "days since last transaction") suddenly exhibits a high velocity of new charges.

Fig 4. Velocity Feature Definition – Amount / Transaction Count

$$\text{Velocity} = \frac{\text{Short-Term Activity}}{\text{Long-Term Activity/Days in Period}}$$

Amount Variability Features

The Amount Variability features quantify the statistical deviation of a current transaction amount from an entity's historical spending distribution. Rather than comparing the current transaction to a single aggregate (such as a simple mean), this metric analyzes the individual "deltas" (differences) between the current transaction and every preceding transaction within a specific temporal window. Then, the differences are aggregated into Mean, Maximum, and Median.

Cross-Entity Diversity Features

Cross-entity diversity features were constructed by counting the number of distinct values of one entity observed within the time windows of another entity. For example, the number

of unique merchants visited by a given card within the prior 7 days, or the number of unique states associated with a merchant number within the prior 30 days. This approach captures the breadth and diversity of transactional behavior across entity pairings. Features were computed for all ordered entity pairs across time windows of 1, 3, 7, 14, 30, and 60 days.

Business Logic Features

A set of domain-specific binary flags and derived scores were constructed to encode fraud indicators grounded in empirical knowledge:

1. **Geographic distance** — The geodesic distance in miles between the ZIP code of the current transaction and the immediately preceding transaction on the same card, computed using centroid coordinates from a ZIP code reference database. Transactions exceeding 1,000 miles from the prior transaction were additionally flagged as potentially indicative of card-present fraud or account compromise.
2. **Amount category** — Transaction amounts were binned into quintile-based categories using a historically expanding distribution estimate, updated in chunks of 5,000 records to approximate a leakage-free ordinal encoding of amount size.
3. **Foreign ZIP code flag** — A binary indicator identifying transactions at merchants whose ZIP code does not correspond to any domestic U.S. ZIP code in the reference database.
4. **Weekend flag** — A binary indicator for transactions occurring on Saturday or Sunday.
5. **Rounded amount flag** — A binary indicator for transactions with whole-dollar amounts, which may reflect structured or manually entered transactions associated with fraudulent or test activity.
6. **High amount for merchant flag** — A binary indicator set when the current transaction amount exceeds twice the merchant's expanding historical average, computed using only prior transactions to maintain temporal integrity.
7. **Merchant dominance score** — For each card, the proportion of its total prior transactions attributable to its single most frequently visited merchant. This score captures the degree to which a cardholder's transaction history is concentrated at a dominant merchant, with deviations potentially indicating abnormal usage patterns.

8. **State inconsistency flag** — A binary indicator identifying instances where a card is used in a different state within the same calendar day as a prior transaction, flagging transactions that would require physically impossible or highly improbable travel.

Feature Selection

Methodology

Our feature selection process is a two-stage approach. First, we apply a univariate filter to rank each variable's individual predictive power. We use the **Kolmogorov-Smirnov (KS) statistic**, which is a measure that quantifies **how well a single variable can separate** our "fraud" and "non-fraud" transactions. A higher KS score means the variable is more effective at distinguishing between these two groups. The variables are ranked by their KS scores, and the **Top 1375** are selected. This number is primarily determined as being around ~14% of the total number of features generated.

Second, this set of 1375 variables are passed into a **Stepwise Feature Selection (SFS)** process, implemented using the `mlxtend` library. A fast, simple, non-linear model, specifically a **LightGBM Classifier** (`LGBMClassifier(n_estimators=20, num_leaves=4)`), is used as the estimator to evaluate feature subsets efficiently.

To provide more flexibility for the final modeling stage and avoid being overly conservative, we ultimately retain a slightly larger set of variables than the SFS algorithm's strict recommendation. This ensures potentially valuable predictors are not prematurely eliminated.

Final Variable List

Below is a list of the final 18 variables selected for the subsequent model building procedure, as well as their corresponding descriptions.

1. **Card: Unique States (1-Day)** — Number of distinct states in which the card was used within the past day. A sudden spike in geographic reach signals potential card compromise.
2. **Card × Merchant × State: 14-Day Spend** — Total amount spent by the card at a given merchant description within a given state over the past 14 days. Captures concentrated regional spend patterns.
3. **Card × Day-of-Week: 1-Day vs. 30-Day Transaction Velocity** — Ratio of the card's transaction count on the prior day to its 30-day count on the same day of the week, normalized per day. Detects short-term spikes relative to the card's typical weekday behavior.

4. **Merchant: Same-Day Spend (1-Day)** — Total transaction amount processed by a specific merchant in the past day. A sudden surge at a merchant may indicate coordinated fraud activity.
5. **Card × ZIP Region: 30-Day Spend** — Total amount spent by the card within a 3-digit ZIP region over the past 30 days. Tracks regional spending concentration.
6. **Card × Merchant × ZIP: 60-Day Maximum Amount** — Largest single transaction amount recorded for this card at this merchant-ZIP combination over the past 60 days. Establishes a historical ceiling for the card-merchant-location context.
7. **Card × Day-of-Week: Unique Merchants (1-Day)** — Number of distinct merchants visited on the current day of the week within the past day. Flags unusual breadth of merchant activity on a given weekday.
8. **Card × Merchant × ZIP: Amount vs. 60-Day Median Ratio** — Ratio of the current transaction amount to the median historical amount for this card at this merchant-ZIP combination over the past 60 days. Identifies whether the current amount is atypically large or small.
9. **Card × Merchant × State: 1-Day vs. 60-Day Spend Velocity** — Ratio of the card's same-merchant-state spend in the past day to its 60-day total, normalized per day. Detects a sudden concentration of spend in a merchant-state context relative to long-run patterns.
10. **Card × Day-of-Week: Same-Day vs. 30-Day Velocity (Squared)** — Same-day transaction count velocity for this card's day-of-week pattern, with the 30-day window squared in the denominator to more aggressively amplify recent bursts of activity.
11. **Card × Day-of-Week: Same-Day vs. 30-Day Velocity** — Ratio of the card's transaction count today to its 30-day count on the same day of the week, normalized per day. Baseline velocity measure for same-day weekday activity.
12. **Card × Day-of-Week: 1-Day vs. 30-Day Velocity (Squared)** — Prior-day transaction count velocity for this card's day-of-week pattern with squared normalization, emphasizing recency more aggressively than the standard ratio.
13. **Card × Day-of-Week: Same-Day Velocity-to-Recency Ratio** — Same-day weekday velocity ratio further divided by the number of days since the card's last transaction. Simultaneously captures transaction burst intensity and how recently the card was active.
14. **Card × Day-of-Week: 1-Day Velocity-to-Recency Ratio** — Prior-day weekday velocity ratio divided by days since the card's last transaction. Combined signal of short-term activity rate and recency.

15. **Card: 1-Day vs. 7-Day Velocity-to-Recency Ratio** — The card's 1-day-to-7-day transaction count velocity, normalized by days since the card's last transaction. A high value indicates both recent reactivation and an accelerating transaction rate.
16. **Card × Day-of-Week: Unique Merchant-State Combinations (1-Day**) — Number of distinct merchant-state pairings seen on the current day of the week within the past day. Reflects geographic and merchant diversity in very recent activity.
17. **Card × Day-of-Week: Unique Card-Merchant-State Combinations (3-Day)** — Number of distinct card-merchant-state contexts observed on the current day of the week within the past 3 days. A high value suggests the card is appearing in an unusually wide variety of merchant-location settings.
18. **Card × Day-of-Week: Unique Card-Merchant Combinations (3-Day)** — Number of distinct card-merchant pairings observed on the current day of the week within the past 3 days. Captures short-term merchant diversity relative to typical weekday usage.

Fig 5. Final Selected Variables

Order	Feature Name	KS Value
1	cardnum_unique_count_for_card_state_1	0.5085
2	card_merchdesc_state_total_14	0.1752
3	card_dow_count_1_by_30	0.5161
4	merchnum_desc_total_1	0.1706
5	card_zip3_total_30	0.1784
6	card_merchdesc_zip_max_60	0.1696
7	card_dow_unique_count_for_Merchnum_1	0.4843
8	card_merchdesc_zip_actual/med_60	0.1701
9	card_merchnum_state_total_amount_1_by_60	0.1584
10	card_dow_count_0_by_30_sq	0.5161
11	card_dow_count_0_by_30	0.5161
12	card_dow_count_1_by_30_sq	0.5161
13	card_dow_vdratio_0by30	0.5161
14	card_dow_vdratio_1by30	0.5161
15	Cardnum_vdratio_1by7	0.4956
16	card_dow_unique_count_for_merch_state_1	0.4843
17	card_dow_unique_count_for_card_merchnum_state_3	0.4843
18	card_dow_unique_count_for_card_merch_3	0.4843

Note: the filter score as shown above is the D-statistic from the KS test.

Preliminary Model Exploration

Model Descriptions

Below is a list of 10 different model implementations, from which the model that is most effective at identifying fraudulent transaction is chosen. The success metrics are defined in the subsequent section.

Logistic Regression

Logistic Regression is a foundational statistical model used for binary classification, such as predicting "fraud" vs. "non-fraud." It works by estimating the probability of an outcome based on a set of input variables. It's known for being fast, easy to interpret, and providing a good baseline, though it may not capture highly complex, non-linear patterns as well as tree-based models.

Decision Tree (DT)

A Decision Tree is a simple, flowchart-like model that makes predictions by asking a series of "if-then" questions about the data's features. It splits the data into branches, with each end-point (a "leaf") representing a final decision. While very easy to visualize and understand, a single tree is often prone to "overfitting" (memorizing the training data) and can be unstable.

Random Forest (RF)

A Random Forest is an "ensemble" model that improves upon the single Decision Tree. It works by building a large number of individual Decision Trees during training, each on a different random subset of the data and features. To make a prediction, it takes a "majority vote" from all the trees. This approach significantly increases accuracy and prevents the overfitting seen in single trees.

LightGBM (LGBM)

LightGBM (Light Gradient Boosting Machine) is a high-performance "gradient boosting" model. Boosting works by building models sequentially, where each new model (a small tree) is trained to correct the errors of the previous ones. LGBM is known for being extremely fast, memory-efficient, and highly accurate, especially on large datasets.

LGBM with SMOTE

This isn't a new model, but rather the application of a specific data-handling technique before training the LGBM model. SMOTE (Synthetic Minority Over-sampling Technique) is used to fix class imbalance (e.g., when there are very few fraud cases). It creates new,

"synthetic" examples of the minority class (fraud) to give the model more balanced data to learn from.

LGBM downsampled

This is an LGBM model that was trained on a "sampled" dataset. This is another method to handle class imbalance, but it's more general than SMOTE. It typically means either "undersampling" (randomly removing many non-fraud cases) or "oversampling" (randomly duplicating the few fraud cases) to create a more balanced training set for the model.

LGBM with Jitter-Based Oversampling

An LGBM model is trained with jitter-based oversampling. Rather than synthesizing entirely new samples like SMOTE, jitter augments the minority class (fraud cases) by creating copies with small multiplicative random noise applied to their feature values. This expands the representation of fraud in the training set while keeping each augmented sample close to its original. It is a lightweight and flexible oversampling technique that can help the model learn more robust fraud patterns without introducing the interpolation artifacts that SMOTE can produce.

CatBoost

CatBoost (Categorical Boosting) is another advanced gradient boosting model, similar to LGBM. Its key feature is its exceptional ability to automatically and intelligently handle categorical features (like 'Merch state' or 'ZIP code') without requiring extensive manual pre-processing, which often leads to improved accuracy and simpler model development.

XGBoost (XGB)

XGBoost (Extreme Gradient Boosting) is another powerful and widely-used gradient boosting framework, and a direct competitor to LGBM and CatBoost. It builds trees sequentially to correct prior errors and is renowned for its high predictive accuracy and built-in regularization, which helps prevent overfitting. It has been a dominant model in many machine learning competitions.

Neural Network (NN)

A Neural Network is a model inspired by the structure of the human brain, composed of interconnected layers of "neurons" (nodes). Data is fed into the first layer, passes through one or more "hidden" layers where complex patterns are learned, and produces a final prediction. NNs are very powerful and can capture highly complex relationships, but can also be computationally expensive and difficult to interpret.

Comparative Performance Table

In this section, we seek to compare different variations of the models mentioned above.

In selecting the primary evaluation metric for this model, our focus was placed on the Fraud Detection Rate at 3% (FDR@3%). This metric measures the proportion of all actual fraudulent transactions that are captured within the top 3% of transactions ranked by predicted fraud probability — that is, the transactions a fraud analyst would investigate if reviewing only the highest-risk 3% of the portfolio. Unlike standard Recall, which evaluates performance at a fixed classification threshold, FDR@3% reflects the operational reality of fraud detection, where investigative capacity is finite and resources must be directed toward the highest-risk cases.

This was a strategic business decision driven by the asymmetric costs associated with misclassification in real-time fraud detection. A false negative, where a fraudulent transaction is missed, results in an immediate, direct, and often irreversible financial loss. Conversely, a false positive, where a legitimate transaction is incorrectly flagged, creates customer friction and operational cost, but is considered a more manageable business problem than the direct financial loss from uncaught fraud. Therefore, our priority is to maximize FDR@3% to ensure the highest possible percentage of fraudulent activity is captured within the investigable population.

Each model's score distribution is presented across three distinct datasets:

- The **Training (Trn) set** (blue) is the historical data used to build and train the models.
- The **Testing (Tst) set** (red) is a hold-out set of unseen data from the same time period, used to validate the model's accuracy and generalization.
- The **Out-of-Time (OOT) set** (green) consists of data from a subsequent time period, offering the most critical benchmark of how the model is expected to perform on new, future data.

The model selection process was guided by three key criteria:

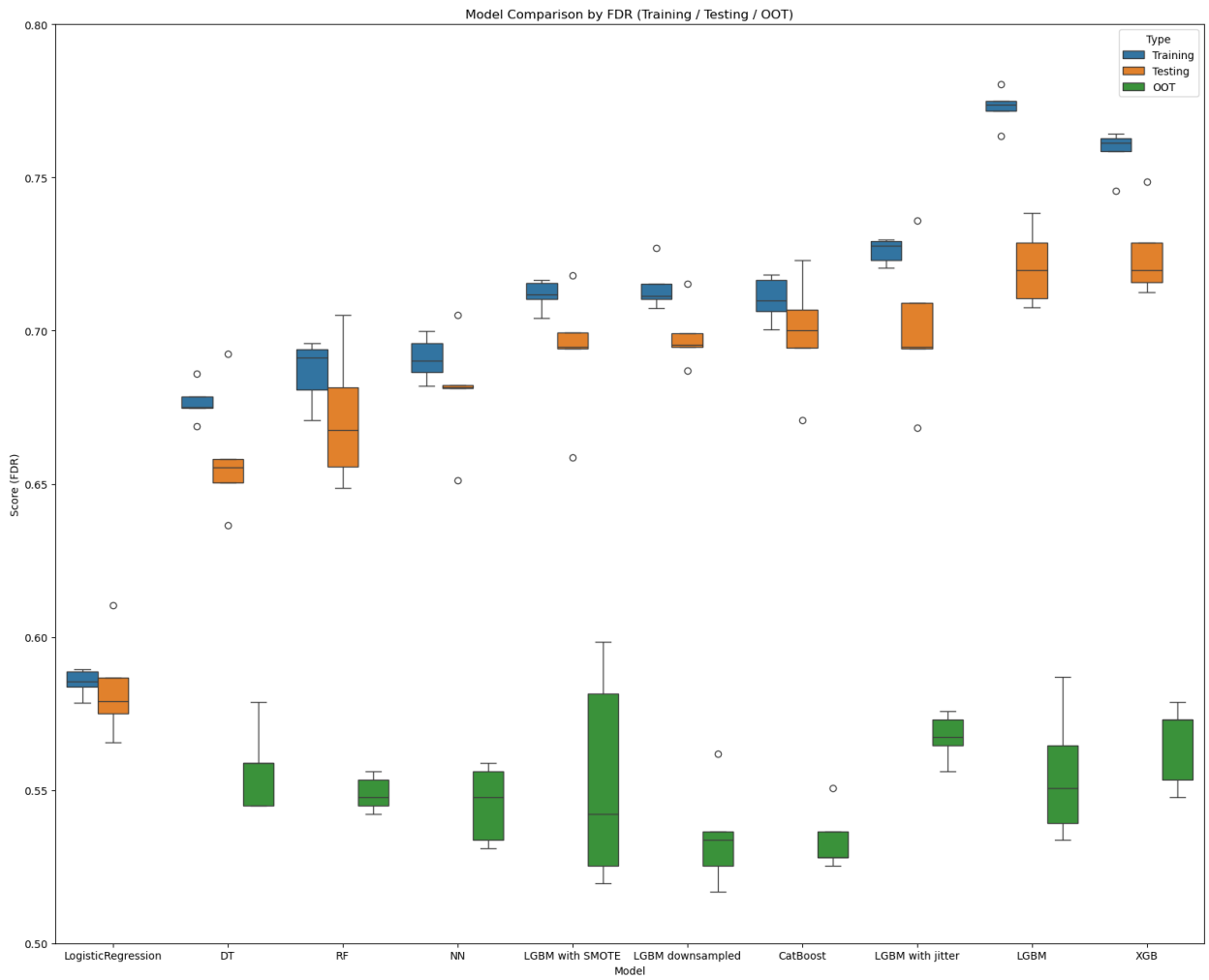
1. **Performance** — Achieving the highest possible FDR@3% on the Testing. The OOT set should always be a final check on the model's performance, but never a benchmark to guide model tuning decisions.

2. **Stability** — Ensuring a minimal gap between Training and Testing scores. A large gap would suggest overfitting, meaning the model has memorized noise in the training data rather than generalizable patterns.

3. **Model Simplicity** — When performance is comparable, preference is given to the less complex model, as it is generally more computationally efficient, easier to interpret, and more robust to maintain.

The following boxplot provides a comparative analysis of ten machine learning model implementations, evaluating their performance based on Fraud Detection Rate (FDR@3%).

Fig 6. Comparison of Performance of Various Models



Final Model Evaluation

Final Model Architecture and Hyperparameters

The final model for this analysis is a Gradient Boosted Tree, implemented via the **XGBoost** framework. This model was selected for its **superior performance on the testing set** and its strong stability, evidenced by a **minimal performance difference** between the training and testing data. The amplitude at which the performance on training over that on testing, a 4.55% improvement, is generally considered very healthy and **does not suggest overfitting.**

This is a powerful ensemble technique that builds a series of decision trees sequentially. Unlike a Random Forest which builds independent trees, this model builds one tree, calculates the errors it made, and then builds the next tree specifically to correct those errors. This process is repeated 80 times, with each new tree incrementally improving the model's overall predictive accuracy.

This specific model uses the 18 features that were selected during the feature selection process. The hyperparameters were tuned to optimize performance and prevent overfitting, as detailed below:

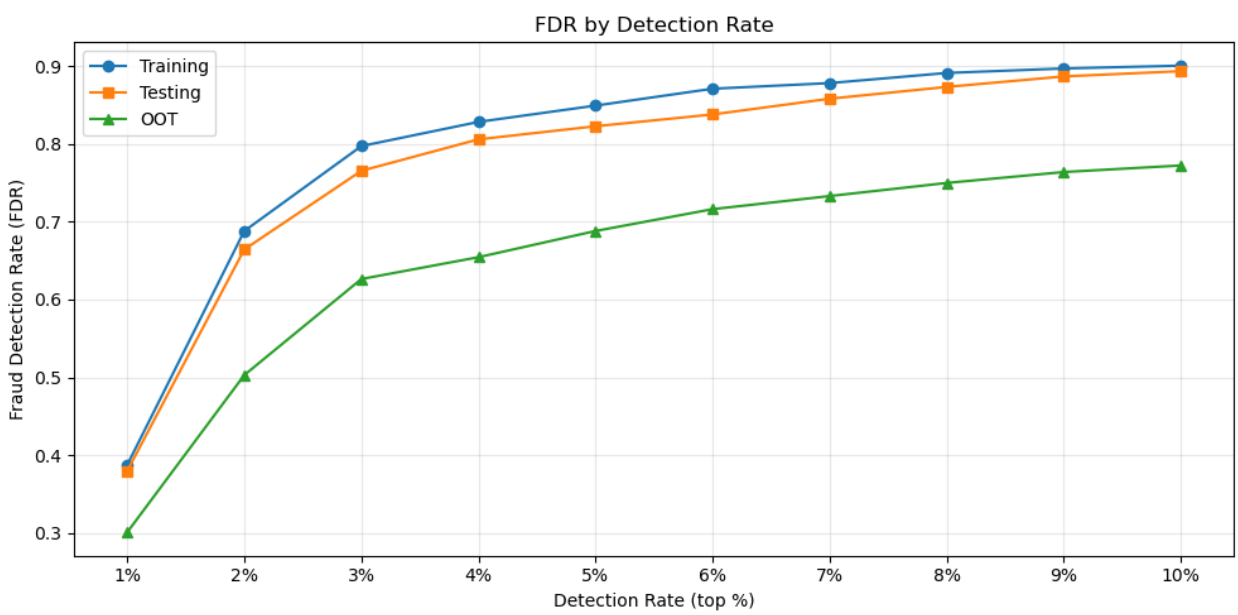
- **`booster = gbtree`**: This confirms the model is tree-based. Each "booster" or "weak learner" in the sequential chain is a decision tree.
- **`max_depth = 4`**: This parameter controls the complexity of each individual tree, limiting it to a maximum of four levels of decision "splits." A depth of 4 was chosen as a balance between capturing complex variable interactions and preventing overfitting. If the trees were too deep, they might "memorize" the training data and fail to generalize to new, unseen transactions.
- **`learning_rate = 0.1`**: This parameter, also known as "eta," controls the "step size" or contribution of each new tree to the final prediction. A value of 0.1 means each tree's correction is scaled down, forcing the model to learn more slowly and conservatively. This helps prevent overfitting and leads to a more robust and stable final model.
- **`n_estimators = 80`**: This defines the total number of trees to be built in the sequence. The model will create 80 trees in total, with each one correcting its predecessor. This number was likely selected through tuning as the point where the model's performance on the test data was maximized.

- `subsample = 0.7`: This is a key anti-overfitting control. Before building each of the 80 trees, the model randomly samples and uses only 70% of the training data. This ensures that each tree learns from a slightly different subset of the data, making the overall model more robust and less likely to be influenced by noise or specific outliers in the training set.

Final Model Performance

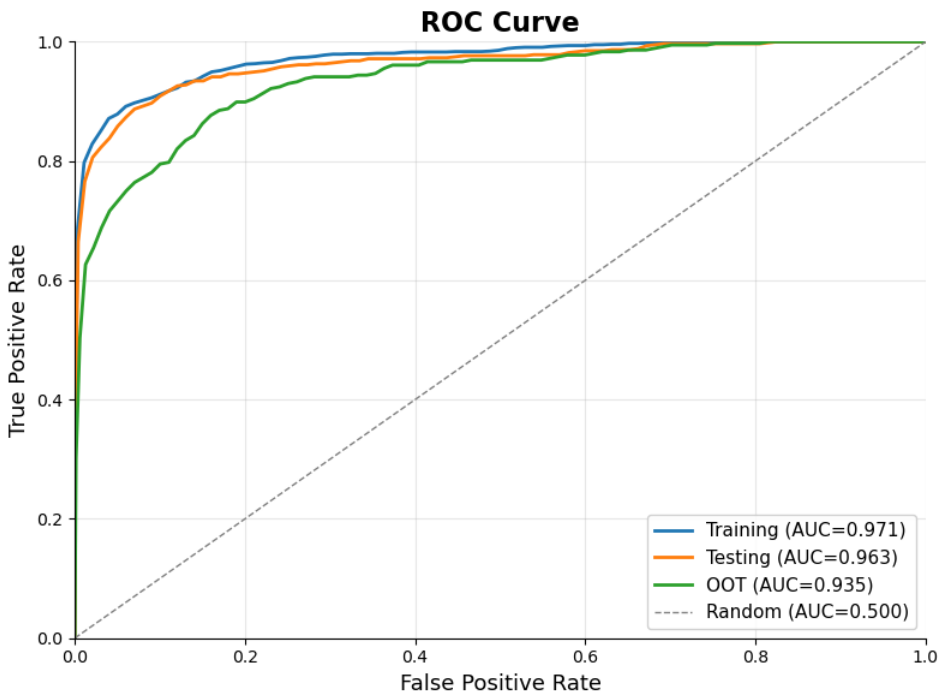
The FDR represents the percentage of total fraudulent transactions correctly identified within a given top percentage of the population ranked by risk score. At the 3% detection rate, the model captures approximately 80% of fraud in the training set and 77% in the testing set, demonstrating strong predictive power and minimal overfitting. While the OOT curve shows an expected performance dip, capturing roughly 63% of fraud at the 3% threshold, it maintains a consistent upward trajectory alongside the other datasets. The model remains robust on the OOT set, successfully identifying nearly 80% of all fraud cases within the top 10% of highest-risk transactions.

Fig 7. Fraud Detection Rate at Top N% Records



The model also demonstrates high discriminatory power, evidenced by Area Under the Curve (AUC) values significantly above the 0.500 random baseline. The **Training (AUC = 0.971)** and **Testing (AUC = 0.963)** curves are closely aligned, suggesting that the model is well-generalized and not suffering from significant overfitting. While a slight performance degradation is observed in the **OOT (AUC = 0.935)** sample, due to temporal shifts in data distributions, the model maintains a robust ability to distinguish between classes.

Fig 8. ROC Curve for Final XGBoost Model



Results: Training Set, Testing Set, Out-of-Time (OOT) Set

Metric Definitions

- **Cumulative % Goods:** This column represents the cumulative percentage of all non-fraudulent ("good") transactions captured, proceeding from the highest-risk bin to the lowest.
- **Cumulative % Bads (FDR):** This is the Fraud Detection Rate (FDR), equivalent to Recall. It measures the cumulative percentage of all fraudulent ("bad") transactions

captured by the model at or above a given risk bin. For example, the top 10 bins (10% of the population) capture 93.5% of all fraud in training set.

- **KS (Kolmogorov-Smirnov):** This statistic identifies the point of maximum separation between the fraudulent and non-fraudulent populations. It is calculated as the difference between the Cumulative % Bads and the Cumulative % Goods at each bin.
- **FPR (False Positive Rate):** This metric represents the False Positive Rate, calculated here as the ratio of cumulative "goods" to cumulative "bads" captured. It quantifies the number of non-fraudulent transactions flagged for every fraudulent one. For instance, at Bin 10, the FPR of 3.22 indicates that 3.22 legitimate transactions are flagged for every 1 fraudulent transaction caught.

Formulas

- Cumulative % Goods = Cumulative Goods / Total # Goods
- Cumulative % Bads (FDR) = Cumulative Bads / Total # Bads
- KS = Cumulative Bads % - Cumulative Goods %
- FPR = Cumulative Goods / Cumulative Bads

Model Performance Tables

The following tables provides a binned performance analysis of the model, ranking all transactions by their predicted fraud score and then grouping them into 100 equal-sized "bins." The analysis is performed on all three data partitions — training, testing, and out-of-time set. The cumulative statistics measure the model's effectiveness as we capture progressively larger percentages of the highest-risk population.

Fig 9a. Final Model Performance on Training Set

	# Records		# Goods		# Bads		Fraud Rate
Training	57,324		55,942		1,382		2.41%
Percentile	Records Reviewed	Cumulative Fraud	Cumulative Good	Cumulative % Bad (FDR)	Cumulative % Goods	KS	FP Ratio
1	573	535	38	38.71	0.07	38.6	0.07
2	1146	951	195	68.81	0.35	68.5	0.21
3	1720	1102	618	79.74	1.1	78.6	0.56
4	2293	1145	1148	82.85	2.05	80.8	1
5	2866	1174	1692	84.95	3.02	81.9	1.44
6	3439	1204	2235	87.12	4	83.1	1.86
7	4013	1214	2799	87.84	5	82.8	2.31
8	4586	1232	3354	89.15	6	83.2	2.72
9	5159	1240	3919	89.73	7.01	82.7	3.16
10	5732	1246	4486	90.16	8.02	82.1	3.6
11	6306	1252	5054	90.59	9.03	81.6	4.04
12	6879	1260	5619	91.17	10.04	81.1	4.46
13	7452	1268	6184	91.75	11.05	80.7	4.88
14	8025	1275	6750	92.26	12.07	80.2	5.29
15	8599	1288	7311	93.2	13.07	80.1	5.68
16	9172	1291	7881	93.42	14.09	79.3	6.1
17	9745	1302	8443	94.21	15.09	79.1	6.48
18	10318	1312	9006	94.93	16.1	78.8	6.86
19	10892	1315	9577	95.15	17.12	78	7.28
20	11465	1320	10145	95.51	18.13	77.4	7.69

Fig 9b. Final Model Performance on Testing Set

Testing	# Records 24,568		# Goods 23,975		# Bads 593		Fraud Rate 2.41%
Percentile	Records Reviewed	Cumulative Fraud	Cumulative Good	Cumulative % Bad (FDR)	Cumulative % Goods	KS	FP Ratio
1	246	225	21	37.94	0.09	37.9	0.09
2	491	394	97	66.44	0.4	66	0.25
3	737	454	283	76.56	1.18	75.4	0.62
4	983	478	505	80.61	2.11	78.5	1.06
5	1228	488	740	82.29	3.09	79.2	1.52
6	1474	497	977	83.81	4.08	79.7	1.97
7	1720	509	1211	85.83	5.05	80.8	2.38
8	1965	518	1447	87.35	6.04	81.3	2.79
9	2211	526	1685	88.7	7.03	81.7	3.2
10	2457	529	1928	89.21	8.04	81.2	3.64
11	2702	532	2170	89.71	9.05	80.7	4.08
12	2948	539	2409	90.89	10.05	80.9	4.47
13	3194	544	2650	91.74	11.05	80.7	4.87
14	3440	549	2891	92.58	12.06	80.5	5.27
15	3685	550	3135	92.75	13.08	79.7	5.7
16	3931	554	3377	93.42	14.09	79.3	6.1
17	4177	554	3623	93.42	15.11	78.3	6.54
18	4422	558	3864	94.1	16.12	78	6.92
19	4668	558	4110	94.1	17.14	77	7.37
20	4914	561	4353	94.6	18.16	76.5	7.76

Fig 9c. Final Model Performance on Out-of-Time Validation Set

	# Records		# Goods		# Bads		Fraud Rate
OOT	12,637		12,281		356		2.82%
Percentile	Records Reviewed	Cumulative Fraud	Cumulative Good	Cumulative % Bad (FDR)	Cumulative % Goods	KS	FP Ratio
1	126	107	19	30.06	0.15	29.9	0.18
2	253	179	74	50.28	0.6	49.7	0.41
3	379	223	156	62.64	1.27	61.4	0.7
4	505	233	272	65.45	2.21	63.2	1.17
5	632	245	387	68.82	3.15	65.7	1.58
6	758	255	503	71.63	4.1	67.5	1.97
7	885	261	624	73.31	5.08	68.2	2.39
8	1011	267	744	75	6.06	68.9	2.79
9	1137	272	865	76.4	7.04	69.4	3.18
10	1264	275	989	77.25	8.05	69.2	3.6
11	1390	278	1112	78.09	9.05	69	4
12	1516	283	1233	79.49	10.04	69.5	4.36
13	1643	284	1359	79.78	11.07	68.7	4.79
14	1769	292	1477	82.02	12.03	70	5.06
15	1896	297	1599	83.43	13.02	70.4	5.38
16	2022	300	1722	84.27	14.02	70.3	5.74
17	2148	307	1841	86.24	14.99	71.3	6
18	2275	312	1963	87.64	15.98	71.7	6.29
19	2401	315	2086	88.48	16.99	71.5	6.62
20	2527	316	2211	88.76	18	70.8	7

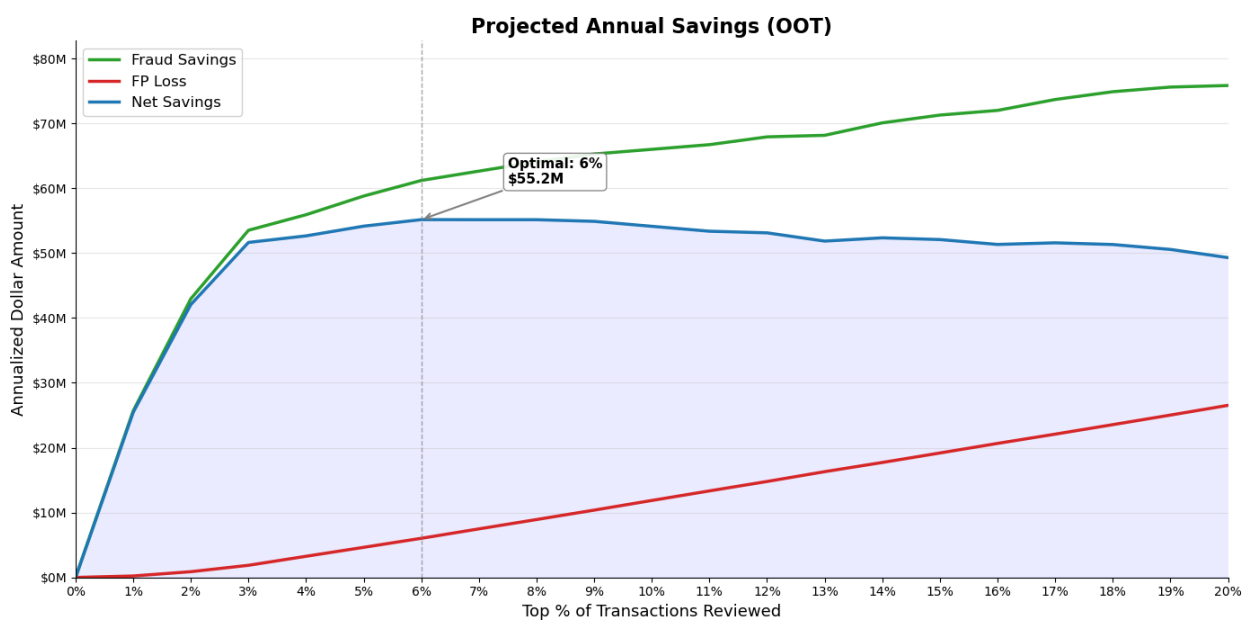
Financial Curves and Recommended Cutoff

Financial Impact

The following plot illustrates the financial trade-off of the model's implementation on the Out-of-Time (OOT) dataset. The x-axis represents the percentage of all transactions flagged (Detection Rate), while the y-axis shows the corresponding dollar impact.

- The **Fraud Savings** (green line) shows the cumulative dollar amount of fraud prevented. It rises steeply at low detection rates, indicating the model successfully identifies high-value fraud first, and then begins to flatten out as most of the fraud is captured.
- The **False Positive Loss** (red line) represents the cumulative cost of reviewing legitimate transactions, which increases steadily as we flag more transactions.
- The **Net Savings** (blue dashed line) is the key business metric, calculated as Fraud Savings minus False Positive Loss. This curve peaks at a **detection rate of approximately 6%**, achieving a maximum net savings of $55.2 million. Beyond this optimal point, the costs incurred from reviewing more false positives begin to outweigh the savings from catching the small amount of remaining fraud, causing the total net savings to decline.

Fig 10. Final Model Performance on Out-of-Time Validation Set



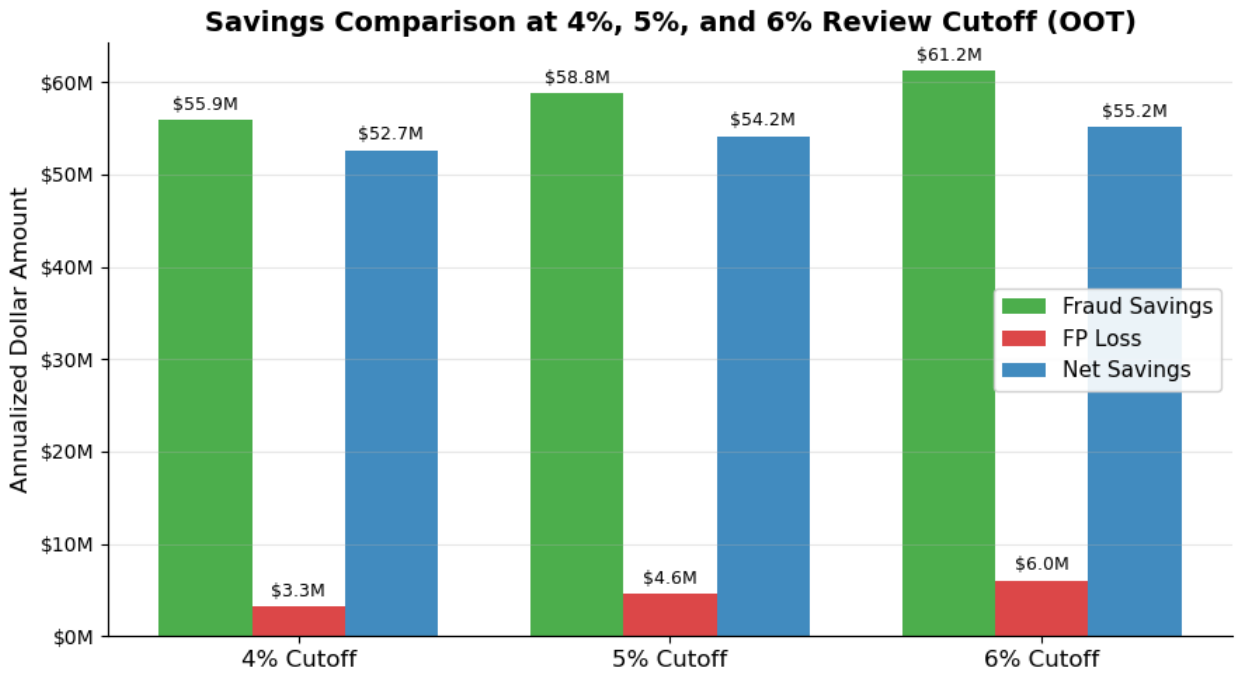
Recommended Cutoff Strategy

As a rule of thumb, we should avoid selecting the theoretical optimal cutoff. False positives carry real costs beyond direct dollar losses — customer friction, complaints, and reputational risk all increase as the FP volume grows, making the true cost harder to quantify and easy to underestimate.

At the same time, we should not cut too close below the optimal point. If our estimate is slightly off, the opportunity cost in missed fraud savings is significant.

The recommended strategy is to implement a **5% detection rate**. Although the theoretical maximum net savings occurs at approximately 6%, analysis indicates **strong diminishing returns** beyond the 5% mark. At this recommended rate, the fraud detection model is projected to yield **$54.2 million in net savings**.

Fig 11. Financials at 4, 5, 6% Review Cutoff



Summary

This report detailed the end-to-end development of a real-time fraud detection model, beginning with data cleaning and exploration of 98,393 transaction records. The process involved a multi-step imputation strategy for missing merchant location data and the creation of rule-based flags for known anomalies, such as recurring FedEx charges. The core of the project was a comprehensive feature engineering process that transformed raw data into powerful predictors. This included creating target-encoded features, time-based aggregations over rolling windows, and velocity and recency ratios designed to detect sudden bursts of activity. Amount variability features and cross-entity diversity features were also created to quantify statistical deviations and track shifts in geographic or merchant breadth. Domain-specific business logic features were also built to capture fraudulent transactions effectively.

A two-stage feature selection process was employed, starting with a Kolmogorov-Smirnov (KS) univariate filter to identify the top ~1400 variables, which were then refined using Stepwise Feature Selection (SFS) to produce a final set of 18 predictors. Ten different machine learning model implementations were explored, with a focus on Fraud Detection Rate @3% as the primary business metric. An XGBoost model was selected as the champion for its superior and stable performance. The final model was tuned with specific hyperparameters (`max_depth=4`, `n_estimators=80`, `subsample=0.7`) to optimize performance while preventing overfitting.

The final model's performance on the critical Out-of-Time (OOT) dataset is highly effective, achieving an **FDR of 62.64% while reviewing only the top 3%** of the highest-risk transactions. A full financial impact analysis was conducted, leading to a recommended business strategy of implementing a **5% detection rate**. This cutoff is projected to yield **$54.2 million in net savings**, representing the optimal balance between maximizing fraud capture and minimizing the operational costs of reviewing false positives.

Limitation

Below is a list of limitations and points of improvement regarding this report.

Transaction-Level Scope

This analysis operates entirely at the transaction level, as the dataset lacks account-level information. In practice, fraud detection benefits significantly from account-level assessment. A single compromised account may generate dozens of fraudulent transactions, and declining at the account level yields compounding savings across all associated activity. Correspondingly, the Fraud Detection Rate metric should ideally be evaluated in conjunction with an account-level analogue, defined as the number of fraudulent accounts identified within the top N% of scores divided by the total number of fraudulent accounts in the portfolio.

Limited Data and Segmentation Opportunities

The dataset contains only one year of card transaction history with a narrow set of fields: card number, merchant information, transaction amount, date, and a binary fraud label. This constrains the analysis in two important ways. First, retaining a true out-of-time validation window while maintaining sufficient training data for the model to learn seasonal patterns required a deliberate tradeoff. Second, the lack of product type, channel, or customer demographic attributes prevents meaningful behavioral segmentation. Segmentation is a common and powerful technique in production fraud systems, where separate models or score adjustments are applied to distinct transaction populations exhibiting materially different fraud profiles.

Target Variable Design

The current model optimizes a binary fraud indicator, treating all fraudulent transactions equally regardless of dollar value. An alternative formulation would weight observations by transaction amount, or discretize loss into bins as an ordinal target, directing the model to prioritize detection of high-value fraud over small-dollar cases.

Advanced Modeling Techniques

One underexplored methodology is iterative hard-example mining: train an initial model, remove records that score either very high or very low (i.e., cases the model classifies with high confidence), and retrain on the remaining ambiguous population. This forces the model to concentrate learning on difficult boundary cases rather than obvious signals, and

can meaningfully improve discriminatory performance such as KS. Iterating this process further refines the decision boundary on the hardest-to-classify transactions.

Pre-Deployment Retraining

Because a production environment was not available for this project, the model could not be validated against live outcome data, the most robust form of performance evaluation. Prior to any real-world deployment, the recommended procedure would be to freeze the selected hyperparameters, retrain on the combined training and testing partitions, and assess whether OOT performance improves. If satisfactory, the model would then be retrained on the full available dataset before deployment. Ongoing monitoring and outcome analysis would be essential to detect model drift over time.

Compliance, Explainability, and Fairness

This report focuses exclusively on predictive performance and financial impact. A production deployment would require substantially more scrutiny on regulatory and ethical dimensions. Explainability frameworks such as SHAP values would be necessary to document how individual variables contribute to each prediction, which is often a requirement in many regulated industries. Fairness testing has not been conducted. It remains unknown whether the selected features introduce discriminatory patterns along demographic lines such as gender, race, or geography. The appropriate remediation process would be to measure disparate impact for each sensitive attribute, identify which features drive the disparity, remove or replace them using the next-ranked candidate from the feature selection output, and retest — iterating until the model meets applicable fairness thresholds.

Appendix

Full Data Quality Report (DQR)

1. Data Description

The dataset is **Card Transactions Data**, which includes merchant identifiers, transaction types, and a binary fraud label. The data came from around 100,000 real U.S. applications **over 8 months**. There are **10 fields** and **98,393 records**.

2. Summary Tables

Appendix I. Numeric Fields Table

Field Name	Field Type	# Records Have Values	% Populated	# Zeros	Min	Max	Mean	Standard Deviation	Most Common
Amount	numeric	98393	100.00%	0	0.01	3102046	424.2909	9922.44	3.62

Appendix II. Categorical Fields Table

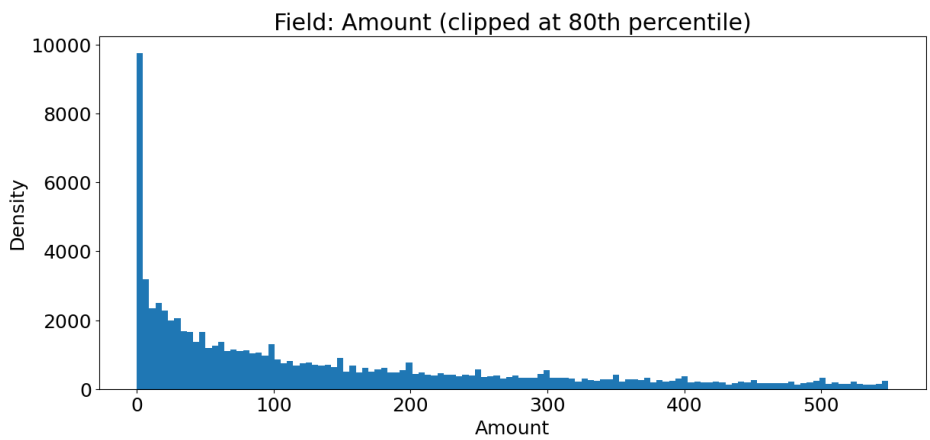
Field Name	Field Type	# Records Have Values	% Populated	# Zeros	# Unique Values	Most Common
Date	Categorical	98393	100.00%	0	365	2/28/10
Merchnum	Categorical	94970	96.52%	0	13091	930090121224
Merch description	Categorical	98393	100.00%	0	13126	GSA-FSS-ADV
Merch state	Categorical	97181	98.77%	0	227	TN
Transtype	Categorical	98393	100.00%	0	4	P
Recnum	Categorical	98393	100.00%	0	98393	1
Fraud	Categorical	98393	100.00%	95901	2	0
Merch zip	Categorical	93664	95.19%	0	4567	38118
Cardnum	Categorical	98393	100.00%	0	1645	5142148452

3. Visualization of Each Field

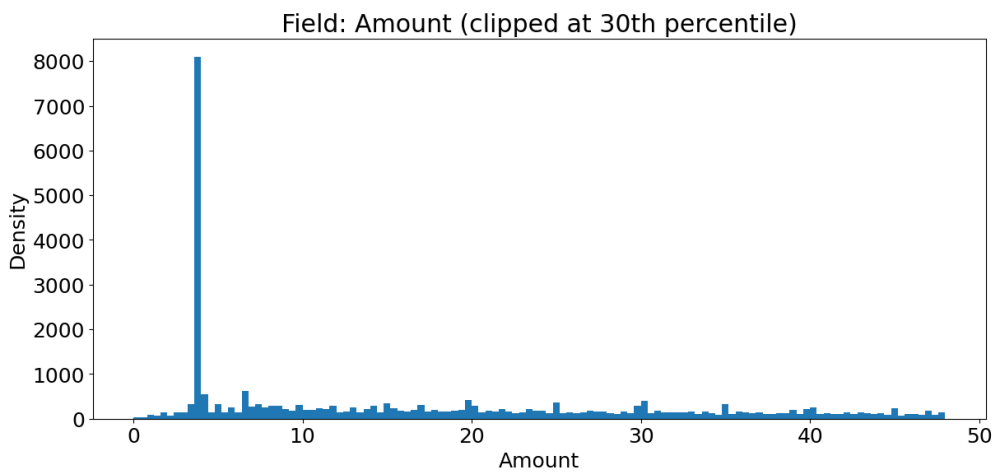
Field Name: Amount

Amount of each transaction. Strictly positive. Many transaction amounts are between $3-$4. When logged, the distribution shows a drop off in number of transactions for amount bigger then ~$2500.

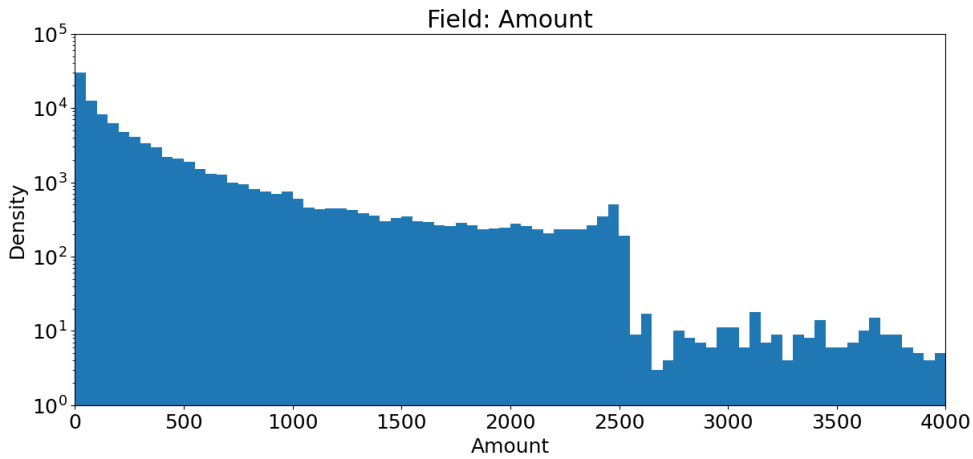
Appendix III. Amount (Clipped at 80th Percentile)



Appendix IV. Amount (Clipped at 30th Percentile)



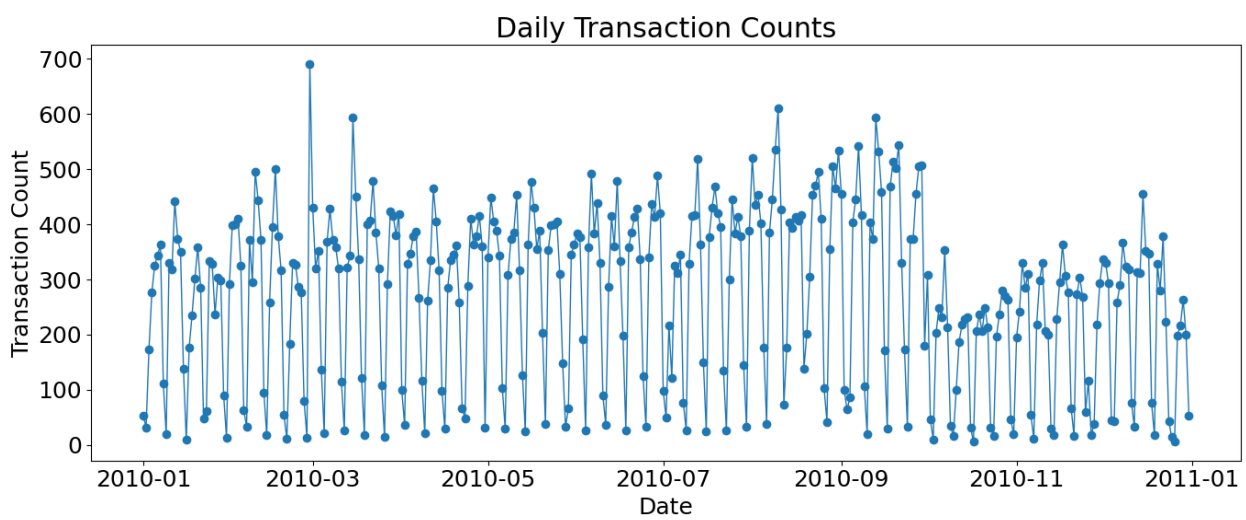
Appendix V. Amount



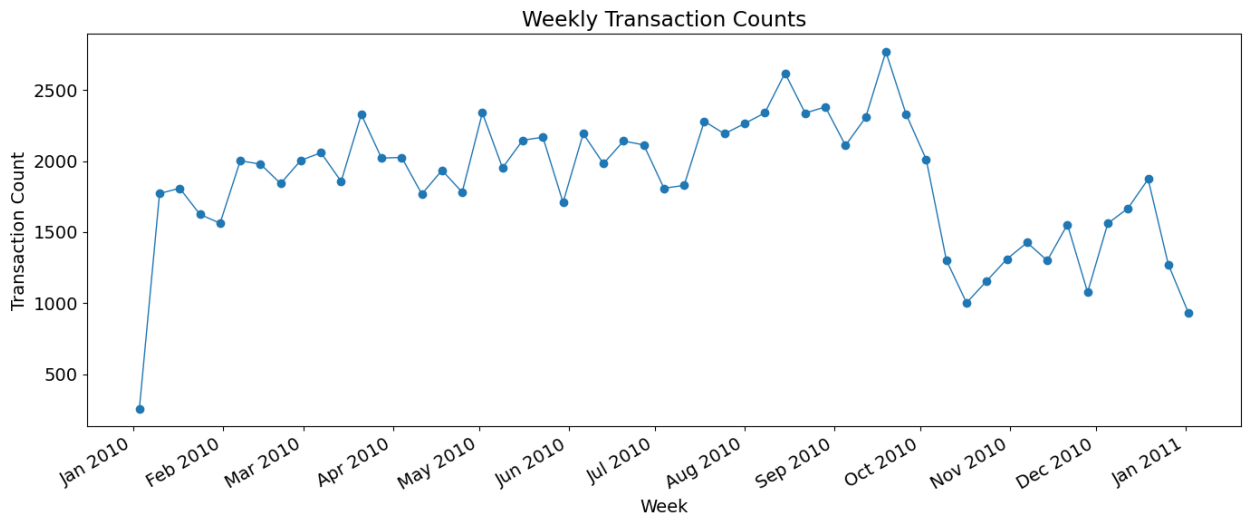
Field Name: date

Transaction date. The first distribution shows the number of daily transactions across time. The second distribution shows the number of weekly transactions across time.

Appendix VI. Daily Transaction Counts



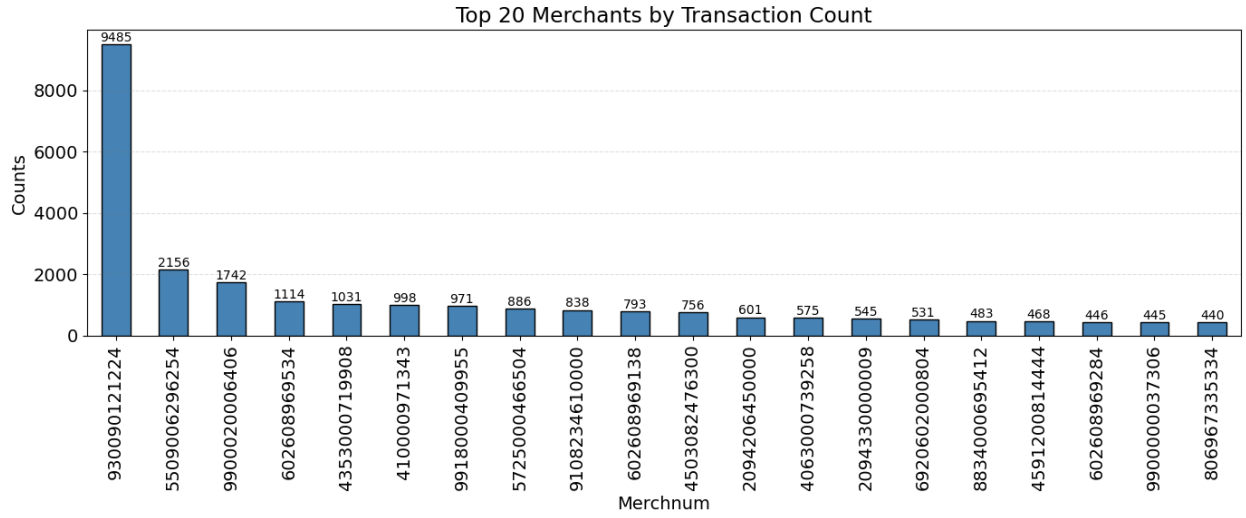
Appendix VII. Weekly Transaction Counts



Field Name: Merchnum

Merchant identification number. The distribution shows the top 20 field values of merchant identification number. The most common ssn is 930090121224, with a total count of 9,485.

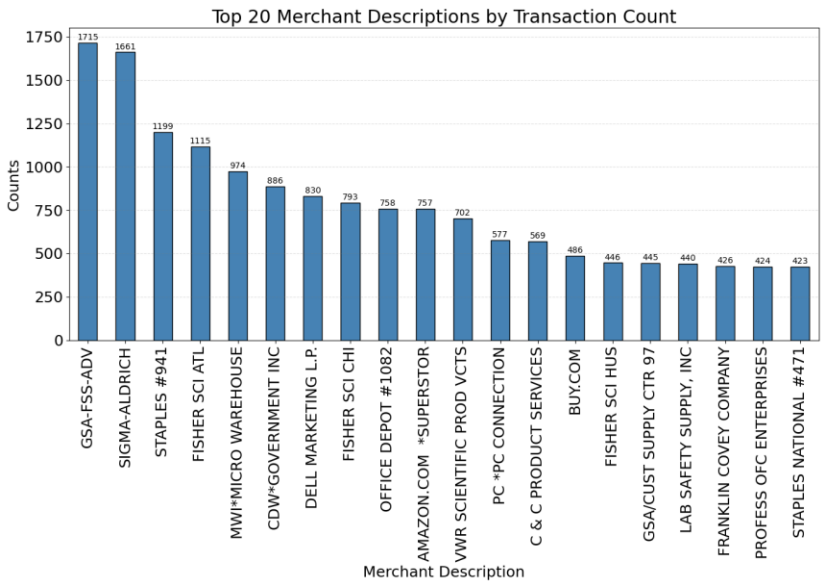
Appendix VIII. Top 20 Merchants by Transaction Count



Field Name: Merchant Descriptions

Description of the merchant. The distribution shows the top 20 field values of merchant description.

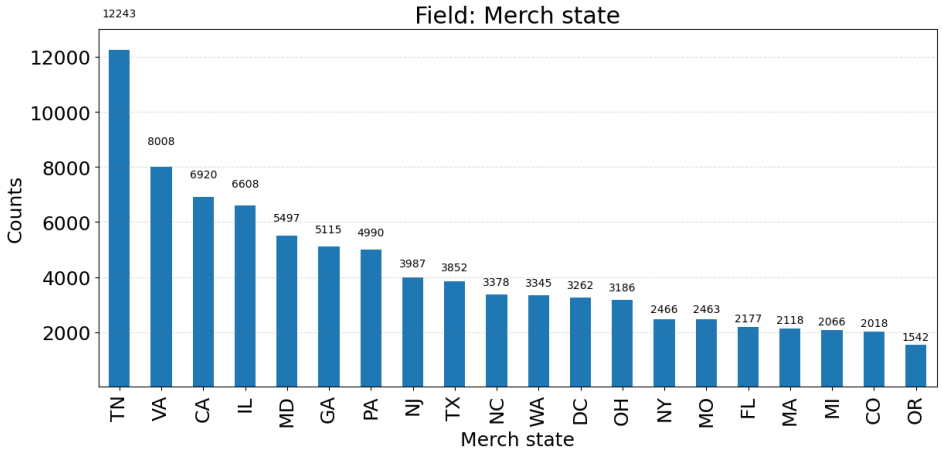
Appendix IX. Top 20 Merchant Descriptions by Transaction Count



Field Name: Merch state

The state that the merchant is in. The distribution shows the top 20 field values of the state that the merchant is in. The most common state is TN.

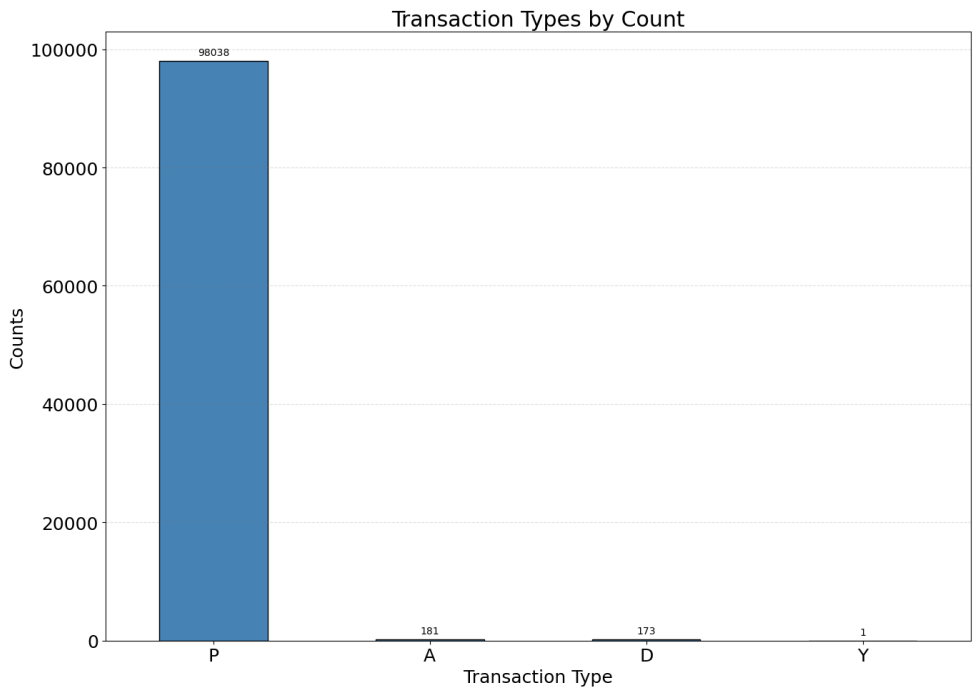
Appendix X. Top 20 Merchant States by Transaction Count



Field Name: Transtype

Transaction Type. Majority of them is purchase and there are not enough data for other types. Consider removing A, D, and Y.

Appendix XI. Top 20 Merchant States by Transaction Count



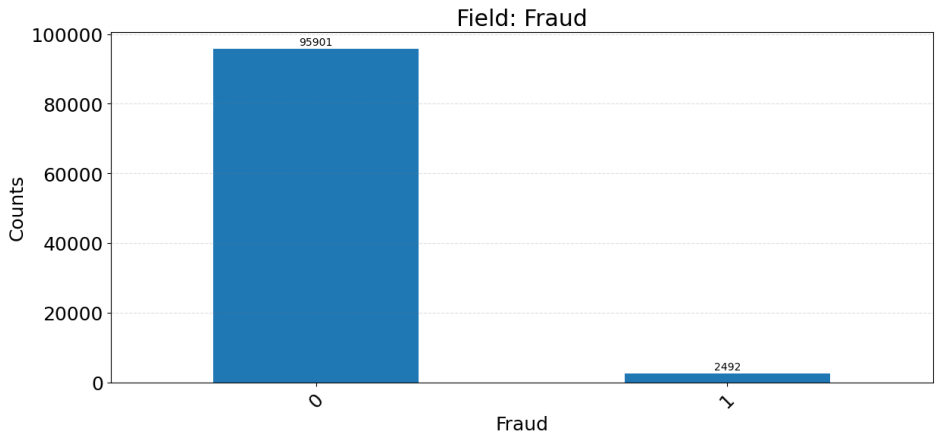
Field Name: Recnum

Recnum is the primary key of the table. Unique value for each row. No nulls.

Field Name: Fraud

Label for fraudulent transaction. 1 for affirmative, 0 otherwise. No nulls.

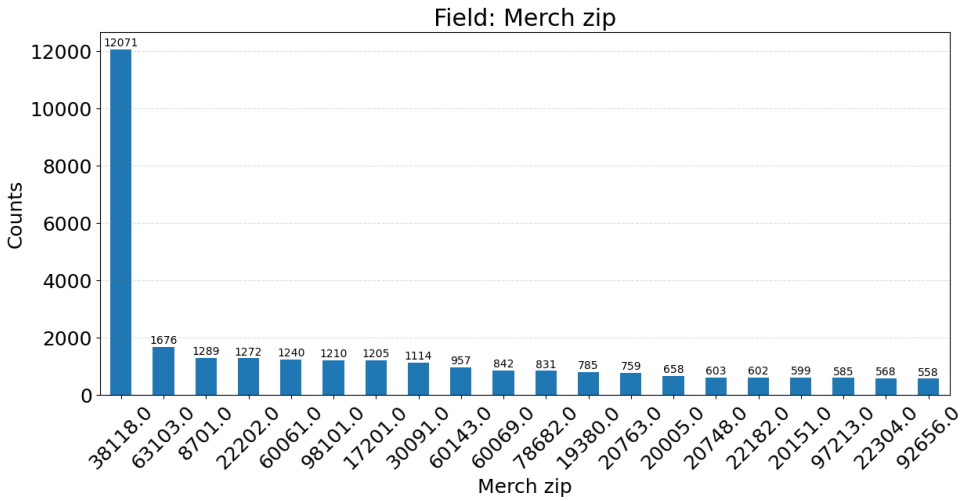
Appendix XII. Counts of Fraudulent vs. Non-Fraud Transactions



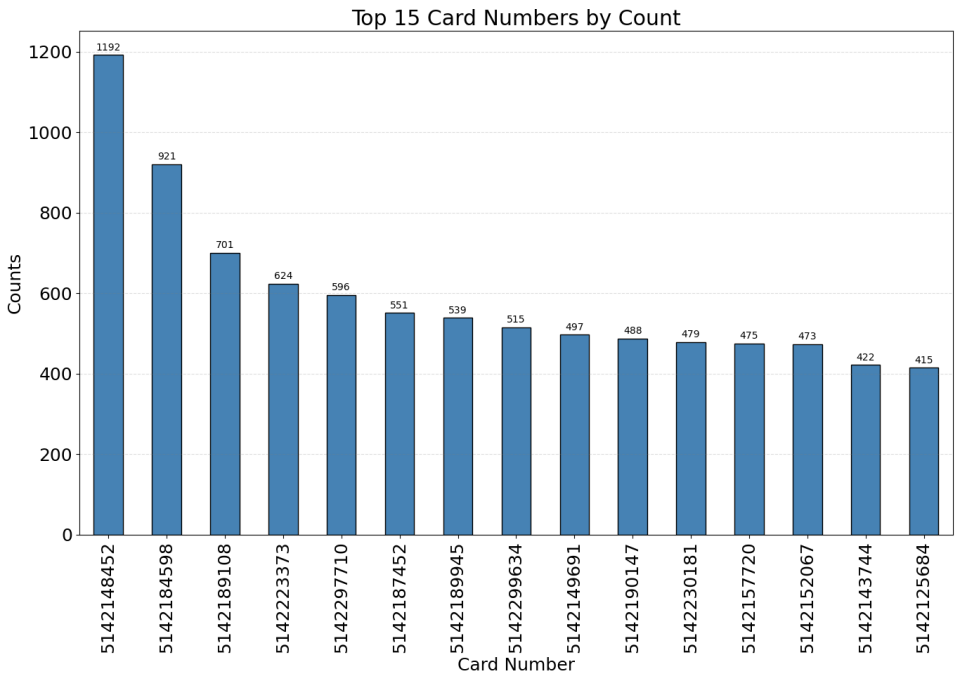
Field Name: Merchzip

Merchant Zip Code.

Appendix XIII. Top 20 Merchant Zip Codes by Transaction Count



Field Name: Cardnum

Card Number. The card number with most transactions are shown below.

Appendix XIV. Top 15 Card Numbers by Transaction Count